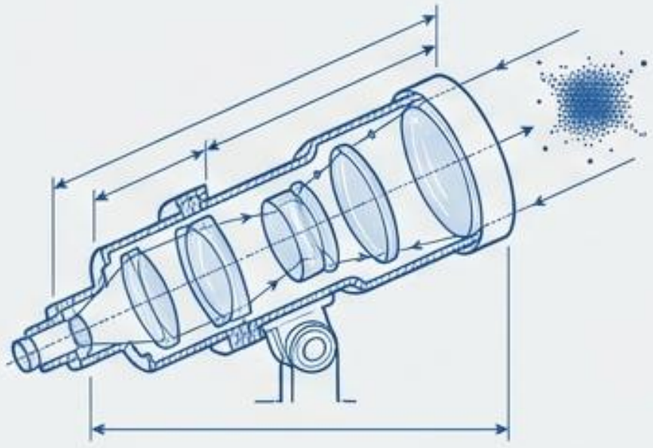
실무 SQL 마스터 클래스: 조회, 성능, 그리고 함수

데이터베이스 엔지니어의 정밀한 설계도 (The Database Engineer's Blueprint)

대상:	주니어 백엔드 개발자, 데이터 분석가, DBA
목표:	실무에서 이 기술들을 언제, 왜 써야 하는가 (Practical Playbook)

NotebookLM

데이터를 다루는 3가지 가상 테이블 기법



저장된 검색 (VIEW)

테이블처럼 보이지만
실제 데이터는 없는
'저장된 SELECT문'

자주 쓰는 SELECT를
이름 붙여 저장해둔 렌즈

즉석 검색 결과 (Inline View)

FROM 절 안에서 한 번 쓰고
사라지는 임시 테이블

중간 계산을 위해
잠깐 쓰는 일회용 컵

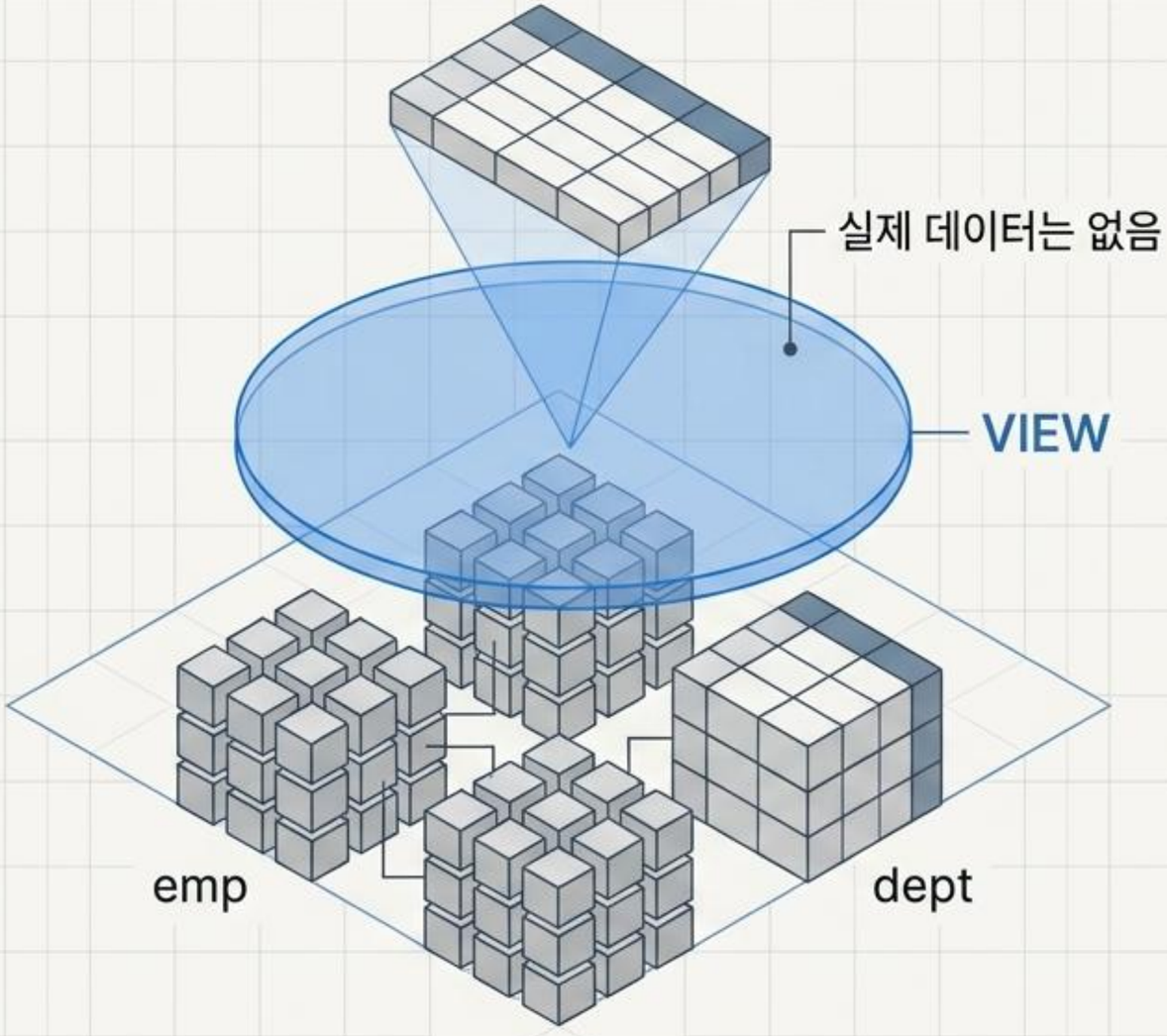
단일 값 추출 (Scalar Subquery)

SELECT 절 안에 들어가
반드시 '하나의 값'만 반환

정확히 값 하나만 콕 짚어
가져오는 스포이드

NotebookLM

VIEW: 복잡성을 숨기는 렌즈



핵심 정의: 실제 데이터 ❌, 쿼리만 저장됨 ⭕.
조회할 때마다 원본 테이블 접근.

실무에서 언제 쓰는가? (Golden Rule)

✔ 여러 사람이 같은 통계 쿼리 반복 사용할 때
✔ 복잡한 JOIN을 숨길 때 (추상화)
✔ 특정 컬럼(급여 등)만 보여주는 보안 목적

```
CREATE VIEW v_emp_info AS
  SELECT e.ename, d.dname, e.sal
  FROM emp e
  JOIN dept d ON e.deptno = d.deptno;

-- 실무 느낌: 매번 JOIN 쓰지 말고 그냥 v_emp_info 써라
```

⚠️ 주의: VIEW는 성능 개선 도구가 아닙니다! (물리뷰 제외)

NotebookLM

인라인 뷰 (Inline View): 일회용 임시 테이블

핵심 정의:

FROM 절 안에 들어가는 SELECT문. 쿼리 단계를 나눌 때 필수.

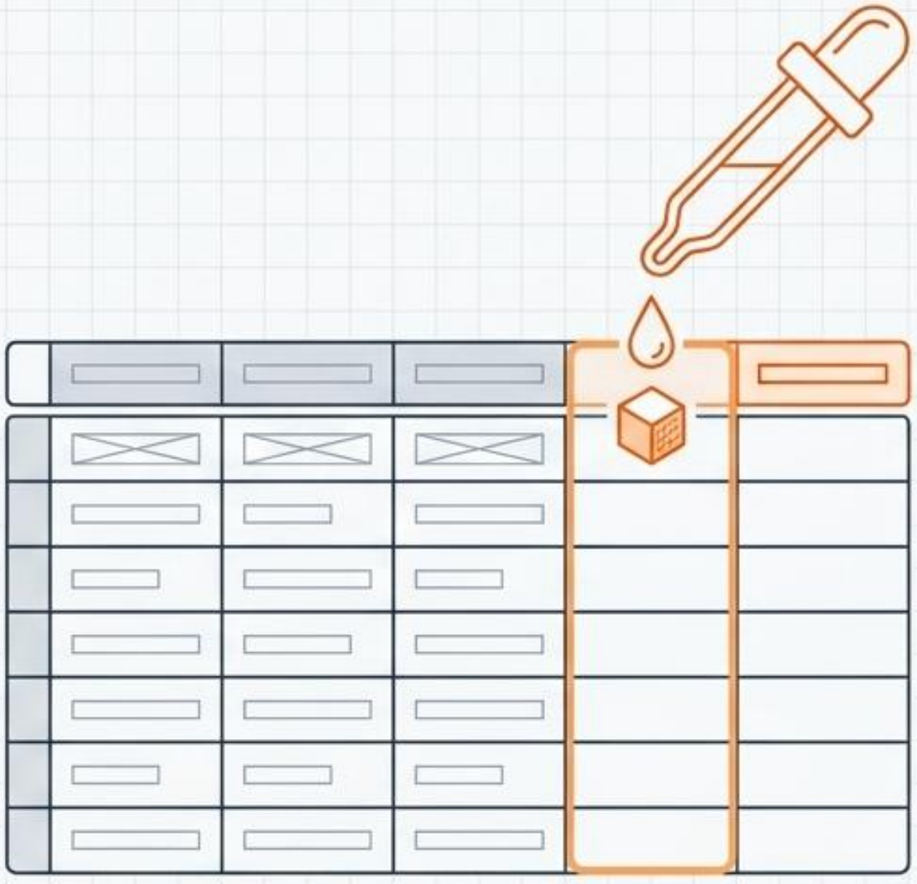
실무에서 언제 쓰는가? (Golden Rule)

- ✔ 한 번만 쓰는 임시 결과가 필요할 때
- ✔ 먼저 집계/정렬하고 나중에 필터링할 때 (TOP-N 쿼리)

```
-- 상위 3명 급여 추출 (먼저 정렬하고, 나중에 자른다)
SELECT *
FROM (
    SELECT ename, sal
    FROM emp
    ORDER BY sal DESC  -- 1단계: 정렬된 임시 테이블 생성
)
WHERE ROWNUM <= 3;     -- 2단계: 필터링
```

NotebookLM

스칼라 서브쿼리 (Scalar Subquery): 값 하나 계산



핵심 정의:

SELECT 안에 들어가 반드시 결과 **1개**(한 행, 한 컬럼)만 반환.

실무에서 언제 쓰는가? (Golden Rule)

✅ 각 행마다 부서명, 평균값 등 **'값 하나'**를 덧붙일 때

✅ JOIN을 쓰기에는 너무 단순한 코드 변환일 때

```
-- 부서별 평균 급여 비교
SELECT ename, sal
FROM emp e
WHERE sal > (
    SELECT AVG(sal)
    FROM emp
    WHERE deptno = e.deptno
);
```

⚠️ **성능 경고:** 스칼라 서브쿼리는 메인 쿼리의 행 수만큼 반복 실행될 수 있습니다. SELECT 절에서 남발하면 성능이 박살납니다!

NotebookLM

[핵심 요약] 가상 테이블 & 인덱스 선택 매트릭스

구분		저장 여부	위치	특징	실무 느낌 (언제 쓰는가)
	VIEW	⭕ DB에 저장	없음	쿼리 재사용	매번 똑같은 조인 치기 귀찮을 때
	인라인 뷰	❌ 저장 안됨	FROM 절	임시 테이블	먼저 정렬하고 나중에 자를 때 (TOP-N)
	스칼라 서브쿼리	❌ 저장 안됨	SELECT / WHERE 절	정확히 값 1개	컬럼 하나만 살짝 추가하고 싶을 때
	INDEX	⭕ 테이블 연결	없음	검색 속도 향상	데이터가 너무 많아서 조회가 느릴 때

NotebookLM

JOIN vs 서브쿼리: 실무 성능의 절대 법칙

JOIN: 한 번에 합쳐서 고속 처리

서브쿼리: 1000번 반복 실행 (느려질 위험)

🎯 **결론 먼저: 웬만하면 JOIN이 정답이다. (데이터가 많고 성능이 중요할 땐 무조건 JOIN)**

- **서브쿼리를 써도 되는 예외 상황 (3가지)**
  1. 결과가 1건임이 확실할 때 (`WHERE sal = (SELECT MAX(sal) FROM emp))`
  2. 집계 함수와 조건 비교할 때 (`WHERE sal > AVG(sal))`
  3. EXISTS를 사용할 때 (JOIN만큼 성능이 좋음)

🔥 **실무 한 줄 요약: JOIN으로 다 풀고, 안 되면 서브쿼리를 써라**

NotebookLM

필수 내장 함수 대시보드 1 (문자 & 숫자)

문자 함수 (Character Functions)

UPPER('abc') → ABC

LOWER('ABC') → abc

INITCAP('hello') → Hello

LENGTH('ABC') → 3

SUBSTR('ABCDEFG', 1, 3) → ABC

INSTR('ABCDEFG', 'C') → 3 (💡 팁: INSTR + SUBSTR 조합으로 문자열 검색)

REPLACE('ABCABC', 'A', 'X') → XBCXBC

TRIM(' ABC ') → ABC

LPAD('A', 5, '*') → ****A

RPAD('A', 5, '*') → A****

숫자 함수 (Number Functions)

ROUND(123.456, 2) → 123.46 (반올림)

TRUNC(123.456, 2) → 123.45 (버림)

MOD(10, 3) → 1 (나머지)

CEIL(3.2) → 4

FLOOR(3.8) → 3

POWER(2, 3) → 8

SQRT(16) → 4

ABS(-10) → 10

NotebookLM

필수 내장 함수 대시보드 2 (날짜, 변환, NULL)

날짜 & 변환 (Date & Conversion)

```
SYSDATE / CURRENT_DATE
ADD_MONTHS(SYSDATE, 3),
MONTHS_BETWEEN(d1, d2)
NEXT_DAY, LAST_DAY
TO_CHAR(SYSDATE, 'YYYY-MM-DD')
TO_DATE('2025-01-01', 'YYYY-MM-DD')
```

NULL 처리 (Handling NULL)

```
NVL(col,0) → NULL이면 0
NVL2(col, 'NOT NULL','NULL')
COALESCE(a,b,c) → 첫 번째 NOT NULL 값 리턴
```

조건 함수 (Conditional)

```
DECODE(expr, val1, res1, default)
CASE WHEN 조건 THEN 값 ELSE 값 END
```

💡 실무 팁

- 날짜를 비교할 때는 TRUNC(SYSDATE)로 시간을 제거한 뒤 비교하라.
- NULL 처리는 NVL보다 여러 인자를 받을 수 있는 COALESCE가 확장성이 좋다.
- DECODE보다 CASE WHEN이 가독성이 월등히 좋고 표준 SQL이다.

NotebookLM

실무자를 위한 3가지 SQL 멘탈 모델 (The Practitioner's Mantra)

Rule 1. (구조)

가상 테이블(VIEW)은 마음껏 쓰되, 서브쿼리는 JOIN으로 먼저 해결하라.

(JOIN은 한 번에, 서브쿼리는 한 줄씩 계산된다)

Rule 2. (성능)

성능 문제의 80%는 INDEX로 해결된다.

(데이터가 느릴 때 가장 먼저 실행 계획과 B-Tree 목차를 확인하라)

Rule 3. (원칙)

인덱스는 공짜가 아니다.

(조회는 빨라지지만 삽입/수정은 느려진다.
함수와 연산으로 인덱스를 깨뜨리지 마라)

NotebookLM